Module 7: Spring Boot Testing — JUnit, Mockito, Integration Testing, and JaCoCo

Questions

Q1. Why is testing important in a Spring Boot application, and what does Spring Boot provide out of the box?

Answer: Testing is important because it catches regressions early, documents expected behavior, and gives confidence during refactoring or production incidents. Spring Boot provides the `spring-boot-starter-test` starter, which bundles JUnit 5, Mockito, AssertJ, Hamcrest, JSONassert, and JsonPath. It also offers slice test annotations such as `@WebMvcTest`, `@DataJpaTest`, and `@RestClientTest`, which load only the relevant slice of the application context so tests run faster than a full `@SpringBootTest`. The starter also configures the build tools to separate unit tests from integration tests, which is useful on CI where you want quick feedback before running slower integration suites.

Q2. What is the difference between unit testing and integration testing in a Spring Boot project?

Answer: A unit test isolates a single class or method and replaces all collaborators with test doubles, so a failure points directly to the unit under test. An integration test loads part or all of the Spring context and exercises real collaborators such as the database, the web layer, or message brokers. Unit tests are fast, deterministic, and run without a server or database. Integration tests are slower but verify wiring, transaction behavior, serialization, and configuration that unit tests cannot. A healthy project runs unit tests on every commit and integration tests in a separate CI stage or nightly job.

Q3. What is JUnit 5, and how is it different from JUnit 4?

Answer: JUnit 5 is the current generation of the JUnit testing framework. It has a modular architecture made of JUnit Platform, which discovers and runs tests; JUnit Jupiter, which provides the modern programming and extension model; and JUnit Vintage, which can run JUnit 3 and 4 tests. Unlike JUnit 4, it uses `@Test`, `@BeforeEach`, `@AfterEach`, `@BeforeAll`, and `@AfterAll` annotations, supports parameterized tests, dynamic tests, and a powerful extension API. The extension model replaces JUnit 4 runners and rules, so Spring uses `SpringExtension` instead of the old `SpringRunner`.

Q4. Explain the lifecycle annotations `@BeforeAll`, `@BeforeEach`, `@AfterEach`, and `@AfterAll` in JUnit 5.

Answer: `@BeforeAll` runs once before any test in the class and is typically used for expensive

setup such as starting a database container. `@BeforeEach` runs before every test and is used to reset state, create fresh mocks, or reinitialize the object under test. `@AfterEach` runs after every test to clean up resources like open files or database rows inserted by the test. `@AfterAll` runs once after all tests complete and is used to stop containers or close connection pools. Static setup methods must be `static`, which is a common trap when you need shared state across tests.

Q5. What is AssertJ, and why do developers prefer it over standard JUnit assertions?

Answer: AssertJ is a fluent assertion library that makes test failures readable and assertions easy to write. Instead of `assertEquals(expected, actual)`, you write `assertThat(actual).isEqualTo(expected)`. It has rich support for collections, exceptions, strings, dates, and JSON, and the failure messages are often clearer than JUnit's defaults. For example, `assertThatThrownBy(() -> service.findById(-1)).isInstanceOf(NotFoundException.class).hasMessageContaining("not found")` reads like a sentence and bundles multiple checks. AssertJ also avoids confusion about the expected-versus-actual parameter order, which is a common source of misleading failure messages.

Q6. How do you write a basic unit test for a Spring Boot service method?

Answer: You test the service class directly without loading the Spring context. You create the service instance yourself or use Mockito's `@InjectMocks`, and you provide its dependencies with `@Mock`. In the test you arrange the inputs and mock behavior, act by calling the service method, and assert the result or verify interactions. For example, if `OrderService` depends on `OrderRepository`, you mock the repository to return an order, call `orderService.findById(1L)`, and assert that the returned order has the expected customer name. Since the Spring context is not loaded, the test starts in milliseconds.

Q7. What is Mockito, and what problem does it solve?

Answer: Mockito is a mocking framework that lets you create test doubles for dependencies so you can isolate the class you are testing. It solves the problem of slow or unavailable collaborators such as databases, external REST services, or message queues during unit tests. With Mockito you can define stubbed responses, verify that certain methods were called, and capture the arguments passed to mocked methods. This keeps unit tests focused, fast, and deterministic. A common mistake is mocking everything including simple value objects, which makes tests brittle and less trustworthy.

Q8. What is the difference between `@Mock`, `@Spy`, and `@InjectMocks` in Mockito?

Answer: `@Mock` creates a fully mocked object where every method returns default values unless stubbed. `@Spy` creates a partial mock that wraps a real instance and calls real methods by default, unless you stub a specific method. `@InjectMocks` creates an instance of the class under test and tries to inject mocks or spies into its constructor, fields, or setter methods. `@InjectMocks` is convenient for service-layer tests, but constructor injection is cleaner because it makes dependencies explicit and avoids reflection-based injection, which can silently fail if field names do not match.

Q9. How do you stub a method with Mockito, and what happens if a stubbed method is called with unexpected arguments?

Answer: You stub a method with `when(mock.method(args)).thenReturn(value)`, or with the BDD-style `given(mock.method(args)).willReturn(value)`. If the method is called with arguments that do not match any stubbing, Mockito returns the default value for the return type, such as `null` for objects, `0` for numbers, or `false` for booleans. This silent default return is a frequent source of confusion because the test may pass even though the mock interaction is wrong. To catch this, use `Mockito.strictness(Strictness.STRICT_STUBS)`, which fails the test if a stubbed mock is called with unexpected arguments or if a stubbed method is never called.

Q10. What is the purpose of `verify()` in Mockito, and when should you use it?

Answer: `verify()` checks that a specific interaction happened with a mock, such as a method being called a certain number of times or with specific arguments. You use it when the side effect of the method under test is more important than the returned value, for example when a service saves a record or sends an email. `verify(repository).save(order)` confirms that persistence happened, while `verify(emailClient, never()).send(any())` confirms that no email was sent for an invalid order. However, overusing `verify()` on every collaborator makes tests brittle; prefer asserting outcomes and verify only meaningful interactions.

Q11. What is `@MockBean`, and when do you use it instead of `@Mock`?

Answer: `@MockBean` is a Spring test annotation that replaces an existing bean in the Spring application context with a Mockito mock. You use it in integration or slice tests where the context is loaded, such as `@WebMvcTest` or `@SpringBootTest`, and you need a real controller but a fake service. `@Mock` is used in plain unit tests where there is no Spring context. A common pitfall is using `@MockBean` in a class with many tests; it triggers context caching invalidation because Spring sees a modified context, which can slow down the whole suite.

Q12. How do you test the persistence layer in Spring Boot, and what does `@DataJpaTest` do?

Answer: The persistence layer is tested with `@DataJpaTest`, which configures an in-memory or real database, loads `@Entity` classes, and creates a default `TestEntityManager` and repository beans. It does not load the full application context, so web components and services are excluded. You write tests that save entities through the repository and assert that custom query methods return the expected results. Because `@DataJpaTest` is transactional, each test runs inside a transaction that rolls back automatically, leaving the database clean. For complex queries, an in-memory database may behave differently from production, so many teams switch to Testcontainers with the real database vendor.

Q13. What are Testcontainers, and why would you use them for repository tests?

Answer: Testcontainers is a Java library that starts lightweight, throwaway Docker containers for databases, message brokers, and other services during tests. For repository tests, it lets you run PostgreSQL, MySQL, or Oracle inside a container instead of relying on H2. This eliminates subtle differences in SQL dialect, transaction behavior, and vendor-specific features between the test database and production. The container starts once per test class, tests run against the real database, and the container is stopped after the tests finish. The trade-off is slower startup, but the confidence gained is worth it for queries that use native SQL, JSON columns, or stored procedures.

Q14. How do you configure Testcontainers for a Spring Boot `@DataJpaTest`?

Answer: You define a container as a static field annotated with `@Container`, start it in `@BeforeAll`, and set Spring DataSource properties so the test context connects to the running container. With the latest Spring Boot and Testcontainers integration, you can also use `@ServiceConnection` so Spring Boot automatically resolves the connection details from the container. You add the database driver and Testcontainers dependency, and you may reuse the same container across tests with a singleton pattern or the Testcontainers reuse feature to reduce startup time. The key is to keep container lifecycle separate from the test transaction so the container keeps running while individual tests still roll back.

Q15. What is `@WebMvcTest`, and what does it load?

Answer: `@WebMvcTest` is a slice test annotation that loads only the Spring MVC web layer. It auto-configures `MockMvc`, registers `@Controller` and `@ControllerAdvice` beans, and excludes the full service, repository, and component scan unless explicitly imported. It is ideal for testing REST controllers because you can simulate HTTP requests, validate status codes, response bodies, and exception handling without starting an embedded server. Collaborating services are provided with `@MockBean`. A common mistake is expecting security filters to be fully configured;

while Spring Security filters are registered, authentication must be mocked or disabled depending on the test intent.

Q16. What is `MockMvc`, and how does it differ from `TestRestTemplate` or `WebTestClient`?

Answer: `MockMvc` is a Spring testing utility that dispatches HTTP requests through the DispatcherServlet without starting a real servlet container. It tests controller behavior, binding, validation, exception handling, and filter interactions in a lightweight way. `TestRestTemplate` and `WebTestClient` are used with `@SpringBootTest(webEnvironment = RANDOM_PORT)` to send real HTTP requests to an embedded server. `MockMvc` is faster and more focused, while the real-client approach verifies the full stack including actual serialization and container filters. For most controller tests, `MockMvc` is sufficient; reserve the real server approach for integration tests that need end-to-end HTTP semantics.

Q17. What is `@SpringBootTest`, and what are its web environment options?

Answer: `@SpringBootTest` loads the complete Spring Boot application context and is used for full integration tests. Its `webEnvironment` attribute controls how the web layer is started. `MOCK` loads a web context but does not start a real server, `RANDOM_PORT` starts an embedded server on a random port, `DEFINED_PORT` uses the configured `server.port`, and `NONE` does not start a web environment. `RANDOM_PORT` is the safest choice for integration tests because it avoids port conflicts on CI and lets `TestRestTemplate` or `WebTestClient` send real requests. `MOCK` is useful when you want the web beans configured but prefer `MockMvc`.

Q18. What is the Spring `TestContext` framework, and how does caching affect test performance?

Answer: The Spring `TestContext` framework manages the application context for tests, including dependency injection, transaction management, and context caching. When multiple test classes use the same context configuration, Spring reuses the cached context instead of creating a new one, which saves significant startup time. However, adding or replacing beans with `@MockBean`, `@SpyBean`, or different `@TestPropertySource` values changes the context key and forces a new context to load. A suite that uses many different `@MockBean` combinations can suffer from context cache pollution and slow builds, so it is worth grouping tests that share the same context.

Q19. How do you test an asynchronous method in Spring Boot?

Answer: Asynchronous methods are usually annotated with `@Async` and executed by a `TaskExecutor`. In a unit test, you call the method and use `CompletableFuture.get()` or

`Future.get()` with a timeout to wait for the result. In an integration test, you may inject the executor and wait for active tasks to finish, or use `Awaitility` to poll until the expected side effect occurs. A common mistake is asserting immediately after the async call, which usually fails because the task has not run yet. You should also test the error path by verifying that exceptions are handled by the returned `Future` or by an `AsyncUncaughtExceptionHandler` for `void` async methods.

Q20. What is JaCoCo, and what does it measure?

Answer: JaCoCo is a Java code coverage library that instruments bytecode during test execution and reports how much of the code was executed. It measures instruction coverage, branch coverage, line coverage, method coverage, and class coverage. The HTML report shows which lines were covered and which branches were missed, while the XML report is consumed by CI tools like SonarQube. Coverage is a useful signal but not a goal by itself; 100 percent line coverage with weak assertions is misleading because the code can be exercised without being verified. JaCoCo should be paired with meaningful assertions and reviewed alongside the test suite quality.

Q21. How do you exclude classes or methods from JaCoCo coverage?

Answer: You configure exclusions in the JaCoCo Maven or Gradle plugin using patterns such as `**/config/*`, `**/dto/**`, or `**/*Exception`. Lombok-generated methods can be excluded with the `lombok.addLombokGeneratedAnnotation = true` setting, which marks generated code so JaCoCo ignores it. Exclusions are useful for boilerplate such as equals and hashCode, mapping classes, or generated sources, but excluding real business logic to hit a coverage target is a harmful practice. The goal is to reduce noise, not to game the metric.

Q22. What are parameterized tests in JUnit 5, and when are they useful?

Answer: Parameterized tests let you run the same test logic with multiple sets of inputs. You can use `@ValueSource` for simple values, `@CsvSource` for tabular data, `@MethodSource` for complex objects, or `@ArgumentsSource` for custom suppliers. They are useful for validation rules, pricing calculations, or any logic that must behave consistently across many cases. For example, a single parameterized test can check that a discount calculator returns the correct amount for customer tiers GOLD, SILVER, and BRONZE. Without parameterized tests, you would copy-paste nearly identical test methods, which is hard to maintain.

Q23. What is the difference between `@MockBean` and `@SpyBean` in Spring tests?

Answer: `@MockBean` completely replaces a bean in the Spring context with a Mockito mock, so calls return stubbed values by default. `@SpyBean` wraps the real bean, so real behavior runs unless a specific method is stubbed. Use `@MockBean` when the real implementation should not execute,

such as an external payment gateway or a repository that is not needed for a controller test. Use `@SpyBean` when you want to verify that the real bean was called or when you need to override one method of an otherwise real service. Spies are useful but can make tests harder to debug because the real code runs and may trigger unexpected side effects.

Q24. How do you handle test data setup in integration tests without making tests depend on each other?

Answer: Each test should start from a clean, well-defined state. Use the transactional rollback behavior of `@Transactional` on tests so database changes are undone. For data that must survive across transactions, such as when testing a service that calls `@Transactional(propagation = REQUIRES_NEW)`, use `TestEntityManager` or a custom cleanup in `@AfterEach` to delete rows. Avoid relying on data created by a previous test because it makes execution order matter and causes flaky failures. Some teams use SQL scripts or libraries like Flyway for baseline schema and small seed data sets, but test-specific data should be created and cleaned up by the test itself.

Scenario based questions

S1. A developer writes a service test that passes locally but fails on CI because of timezone differences. How do you investigate and fix it?

Answer: I first look at any date or timestamp assertions in the test and in the production code. If the test creates an expected value using `LocalDateTime.now()` or `new Date()` and compares it against a value returned by the service, the comparison will fail when CI runs in a different zone or at a different millisecond. The fix is to avoid comparing against wall-clock time in unit tests; instead, inject a `Clock` bean or use a fixed timestamp provided by the test. For integration tests, I ensure the JVM timezone and database timezone are explicit, for example by setting `-Duser.timezone=UTC` in the CI pipeline and using `TIMESTAMP WITH TIME ZONE` columns. I also check whether the test relies on the current date for filtering and replace it with a controlled reference date.

S2. A `@DataJpaTest` passes on its own but fails when the whole suite runs because of leftover data. What is the cause?

Answer: The cause is usually a test that commits data and does not clean up, or a non-transactional operation that leaks state. `@DataJpaTest` is transactional by default, so most tests roll back, but a test calling a service method annotated with `@Transactional(propagation = REQUIRES_NEW)` commits in a new transaction and leaves data behind. Another cause is a `@Sql` script that inserts shared rows without cleanup, or a test that uses native `JdbcTemplate` without a transaction. I find the offending test by running the suite with logging enabled and inspecting the database state after each test class. The fix is to add explicit cleanup in `@AfterEach`, use unique identifiers per test, or refactor the production code so tests can rely on automatic rollback.

S3. A controller test with `@WebMvcTest` returns a 200 status but the response body is empty. How do you debug it?

Answer: An empty body usually means the controller returns a response object but it is not serialized, or the mock did not return the expected object. I first verify that the controller method is actually returning the object and that `@RestController` or `@ResponseBody` is present. Then I check the mocked service; if `when(service.findById(1L)).thenReturn(order)` uses a different argument than the controller passes, Mockito returns `null` and Jackson serializes an empty body. I also check that the object has getters, because Jackson needs them. Finally, I inspect the `MockMvc` result to see whether an exception was swallowed, and I enable debug logging for `org.springframework.web` to trace the request through the filters.

S4. A service test using Mockito throws an `UnnecessaryStubbingException` after a refactor. What is happening?

Answer: `UnnecessaryStubbingException` is thrown when strict stubbing is enabled and a stubbed interaction was never used during the test. After the refactor, the production code may no longer call the stubbed method, or the test may have stubbed a method that was only relevant to an older implementation. I review the test to remove unused stubs and to confirm that the remaining stubs are actually required by the scenario being tested. If strict stubbing is too noisy for a class with shared setup, I either split the test into smaller focused tests or apply `Mockito.lenient()` only where a shared stub is acceptable. The goal is to keep the test honest about what it is verifying.

S5. An integration test with `@SpringBootTest(webEnvironment = RANDOM_PORT)` fails because the actuator health endpoint returns 404. How do you fix it?

Answer: The actuator endpoint is not loaded because `@SpringBootTest` does not auto-configure actuator unless the actuator starter is on the classpath and the endpoint is enabled. I check that `spring-boot-starter-actuator` is a dependency and that the `management.endpoints.web.exposure.include` property includes `health`. If the test uses `webEnvironment = NONE`, actuator web endpoints are not available, so I switch to `RANDOM_PORT` or `MOCK`. I also verify that the test does not override `management.server.port` to a fixed port that collides with another test. When actuator is intentionally disabled for a test, I either enable it for that test class or assert against an internal service bean instead of the HTTP endpoint.

S6. A test with Testcontainers is extremely slow because the database container starts for every test class. How do you speed it up?

Answer: I start by moving the container declaration to a base class or a shared utility so the same

container instance is reused across multiple test classes. With Testcontainers, I enable the `testcontainers.reuse.enable` flag in `~/.testcontainers.properties` so containers stay alive between test runs during local development. On CI, I ensure Docker layer caching is enabled and that the base image is pulled before the build. I also look for unnecessary `@DirtiesContext` annotations that force the Spring context to restart. If the suite is still slow, I move the fastest unit-like repository tests to H2 and reserve Testcontainers only for tests that exercise vendor-specific SQL or stored procedures.

S7. A developer disables CSRF in a `@WebMvcTest` and now real requests fail with 403 in production. What went wrong?

Answer: The test disabled CSRF to make the mock setup easier, but the developer assumed the controller behavior was the only thing being tested. CSRF is part of Spring Security, and disabling it in tests hides the fact that state-changing browser requests require a valid token. The fix is to configure `MockMvc` with the same security settings used in production and to include CSRF tokens in state-changing requests using `csrf()`. For pure API clients using bearer tokens, CSRF is legitimately disabled, but that decision belongs in the production configuration, not hidden in tests. Tests should mirror production security as closely as possible to catch integration issues early.

S8. A test with `@MockBean` runs fine alone but causes context reloads and slow builds across the suite. How do you address it?

Answer: `@MockBean` changes the Spring context fingerprint, so every unique combination of mocked beans triggers a new context cache entry. If many test classes mock different sets of beans, the cache fills up and contexts are constantly created and destroyed. I address this by grouping tests that need the same mocks into the same test class or by using `@Import` to share a common test configuration that declares the mocks once. For unit tests that do not need the Spring context at all, I replace `@MockBean` with plain Mockito `@Mock` and remove `@SpringBootTest`, which eliminates context reloads entirely. When a slice test such as `@WebMvcTest` is sufficient, I use it instead of the full context.

S9. A repository test using H2 passes, but the same query fails in production on PostgreSQL. Why?

Answer: H2 supports a different SQL dialect than PostgreSQL, and some queries that work in H2 may use syntax, functions, or type casting that PostgreSQL does not accept. Native queries, JSON operators, window functions, and custom types are common sources of mismatch. The fix is to move the test from H2 to a Testcontainers PostgreSQL container so the test database matches production. I also review the query for dialect-specific features and consider using JPQL or the Criteria API for portable logic. If a native query is unavoidable, I write a dedicated integration test against the real database and keep the H2 test only for simple repository methods where dialect differences do not matter.

S10. A developer writes a unit test that calls a private method using reflection. Is this a good practice, and what would you suggest?

Answer: Testing private methods through reflection is usually not a good practice because it couples the test to internal implementation details and makes refactoring painful. Private methods are tested indirectly by testing the public methods that call them. If the private method is complex enough to need its own tests, that is often a sign that it should be extracted into a separate class with public methods that can be tested directly. The exception is legacy code that cannot be refactored safely; in that case, reflection may be a temporary bridge, but the goal should be to refactor the code rather than maintain reflection-based tests.

S11. A code coverage report shows 90 percent line coverage, but a bug still reaches production. How is that possible?

Answer: Line coverage only measures whether a line was executed, not whether it was verified. A test can execute error-handling code without asserting that the correct exception is thrown, or it can call a method with a weak assertion that misses the bug. Coverage also does not measure edge cases, concurrency, or integration failures. I treat coverage as a hygiene metric, not a quality guarantee. I review tests for meaningful assertions, add mutation testing or property-based testing for critical paths, and run integration tests against realistic data. If a bug escapes, I add a regression test that fails before the fix and passes after it.

S12. A test that verifies an email was sent keeps failing intermittently. How do you stabilize it?

Answer: Intermittent failures in asynchronous email sending usually come from timing issues. The test asserts before the background thread has finished, or the mock interaction happens on a different thread than the test thread. I stabilize the test by using `Awaitility` to wait until the mock receives the expected call, or by returning a `CompletableFuture` from the email service so the test can wait for completion. If the email client runs inside an `@Async` method, I inject the `TaskExecutor` and wait for active tasks. I also make the test deterministic by controlling the thread pool or by making the email service synchronous in the test configuration.

S13. A build fails because JaCoCo reports zero coverage for classes generated by MapStruct or Lombok. How do you handle it?

Answer: Generated code should not count against coverage because it is not hand-written business logic. For Lombok, I add `lombok.config` with `lombok.addLombokGeneratedAnnotation = true` so JaCoCo recognizes the `@Generated` annotation and skips those methods. For MapStruct, I configure the JaCoCo exclusion list to ignore the generated mapper implementation classes, for example with `**/mapper/*Impl`. I keep the domain and service classes included in coverage so the report reflects actual code quality. If the team uses other generators, I add their output packages to the exclusions and document why those classes are not measured.

S14. A controller test passes in isolation but fails after adding Spring Security to the project. How do you update the test?

Answer: Adding Spring Security introduces filters that intercept every request, so unauthenticated requests to protected endpoints return 401 instead of 200. I update the test by configuring `MockMvc` with the security filters and either providing test credentials or disabling authentication for the test. For `@WebMvcTest`, I add `@AutoConfigureMockMvc(addFilters = false)` only if the goal is to test controller logic without security, but I prefer to keep security enabled and use `@WithMockUser` to simulate an authenticated user. I also add tests that verify the endpoint correctly rejects unauthenticated requests and enforces role-based access, so security behavior is covered, not bypassed.

S15. A test suite reports `IllegalStateException: Failed to load ApplicationContext` after adding a new dependency. What is your debugging approach?

Answer: I start by reading the full stack trace and the caused-by chain, because the root cause is usually buried several levels deep. Common reasons are a missing property, a bean that fails to initialize, a conditional configuration that activates incorrectly under test, or a dependency that requires a real service not available in the test environment. I run a single failing test with debug logging for `org.springframework.boot` and `org.springframework.test` to see which bean fails. If the new dependency is not needed for the test, I exclude it with `@SpringBootTest(classes = { ... })` or a test-specific profile. If it is needed, I provide a mock or a lightweight implementation with `@TestConfiguration`.